

Card Archive

카드 아카이브

송준석

010 4066 8449

rosaa9979@gmail.com

목차

- 프로젝트 개요 3p
- 게임 소개 4p
- 게임 프레임워크 5p
- 핵심 로직 구현 및 리팩토링 6p
- UI / UX 구조 개편 17p
- 기타 21p
- 성과 및 향후 계획 23p

프로젝트 개요



프로젝트 이름

- 카드아카이브 (블루아카이브 2차 창작 팬게임)

프로젝트 기간

- 기획: 2023.10 ~ 2024.06
- 개발: 2024.06 ~ 2025.11

프로젝트 인원

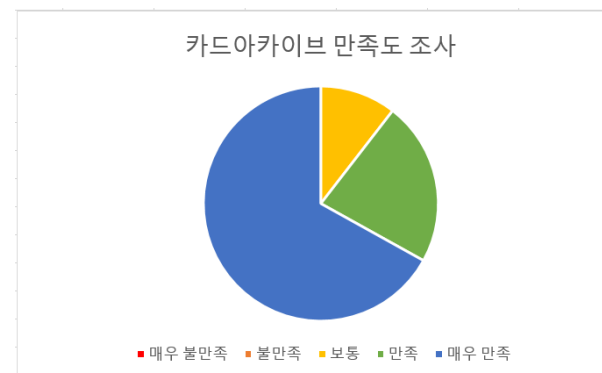
- 1인 프로젝트

성과

- 2025년 10월 일러스타 페스티벌에 출품
- 양일 총 124명이 플레이
- 플레이 후 받은 설문 조사를 통해 게임을 개선



일러스타 페스 부스 사진



만족도 조사

게임 소개 및 규칙 설명 (기본 소개)

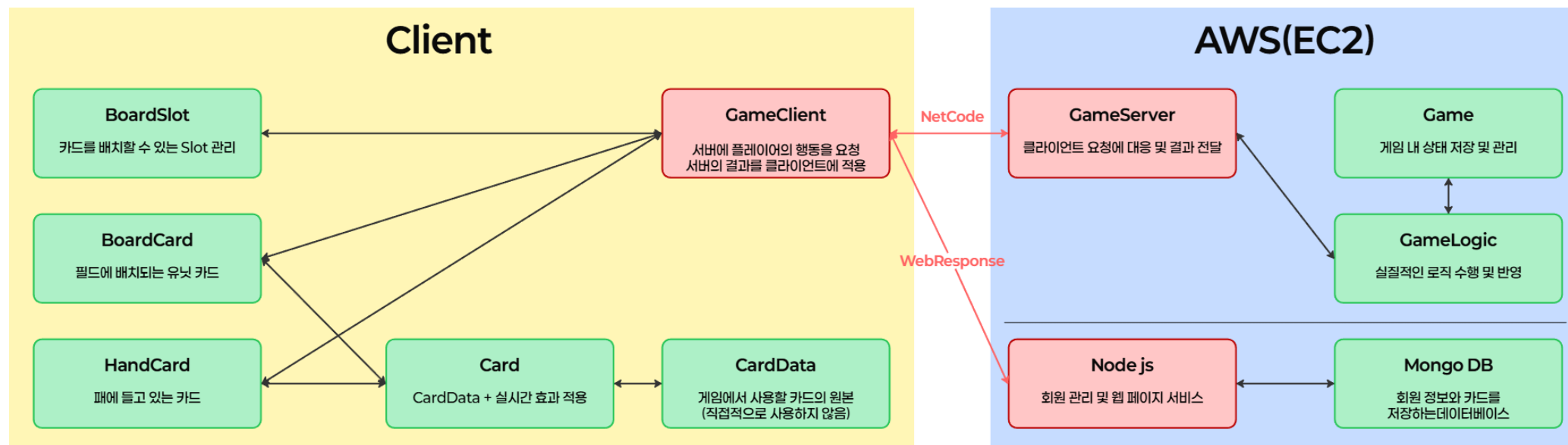
레퍼런스 게임



TEAMFIGHT TACTICS™

여러 시너지를 조합하여 덱을 편성하고
유닛의 시너지와 사거리 및 다양한 카드의 효과를 통해 전장에서 승리하는 TCG 게임

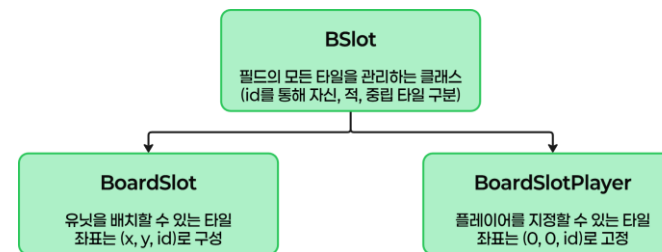
게임 프레임워크



핵심 로직 구현 및 리팩토링 (Slot)

개요

플레이어가 카드를 배치할 때, 배치할 수 있는 타일을 관리하는 클래스



주안점

- 각 플레이어의 입장에서는 아래 진형이 자기 자신의 진형이지만, 서버 입장에서는 누군가는 윗 진형을 선택해야 함
- 즉, 각자가 보는 (1,1)의 좌표가 실제 좌표와 다름



핵심 로직 구현 및 리팩토링 (Slot)

카메라 회전 vs 가상 Slot 클래스

- 카메라 회전 : 게임 시작 시 부여 받은 유저의 id에 따라, 한 쪽 id의 카메라를 반대로 회전한다.
 - 장점 : 플레이어가 보는 좌표와 실제 좌표와 일치함 (좌표 변환 과정 필요 없음)
 - 단점 : Slot이 맡는 역할이 많아져, 유지 보수 및 가독성 어려움, 각 클라이언트의 배경이 달라짐
- 가상 좌표 : BoardSlot과 Slot을 분리
 - BoardSlot : 실제 씬에 배치되는 Slot 클래스이며, GetSlot() 함수를 통해, 실질적인 좌표를 가져옴 (게임 시작 시, 부여 받은 id 값을 통해 한 쪽을 점대칭)
 - Slot : 가상의 Slot 클래스로, 모든 로직은 Slot 기준으로 판단하여 진행함

가상 좌표를 통해 **역할 분리** 및 **유지보수 용이**하도록 설계

BoardSlot은 비주얼 및 연출만 담당하고, 실제 로직은 반드시 Slot으로만 판단

핵심 로직 구현 및 리팩토링 (Ability & Resolve Queue)

개요

Ability : TCG 게임의 메인인 카드 효과 처리를 위한 클래스

Resolve Queue : 발동된 Ability를 적재 및 Dequeue를 통해 적용하는 시스템

주안점

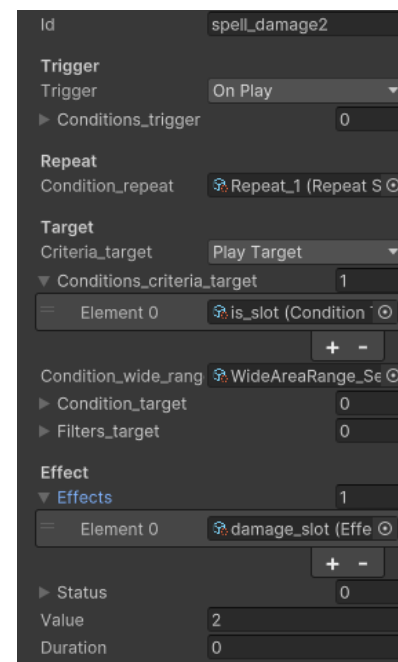
- 다양한 효과를 구현할 수 있게 유지 보수 및 확장성을 고려하여 설계
- 효과를 순서에 맞게 진행하도록 보장

핵심 로직 구현 및 리팩토링 (Ability & Resolve Queue)

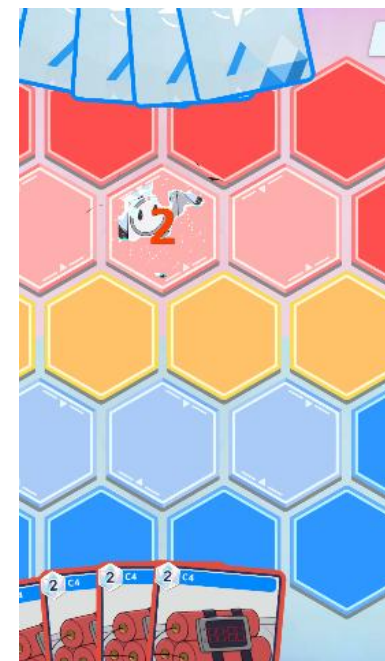
AbilityData 구조

AbilityData	
+ Trigger	발동 시점 정의
+ Trigger Condition	발동 상세 조건 설정
+ Repeat Condition	반복 조건 설정
+ Criteria Target	기준점 정의
+ Criteria Target Condition	기준점 상세 조건 설정
+ WideAreaRange Condition	기준점 기반 범위 설정
+ Target Condition	범위 내 대상 상세 조건 설정
+ Filter Condition	필터링 후 최종 대상 선정
+ Effect	적용할 효과 정의

Ability 클래스 구조



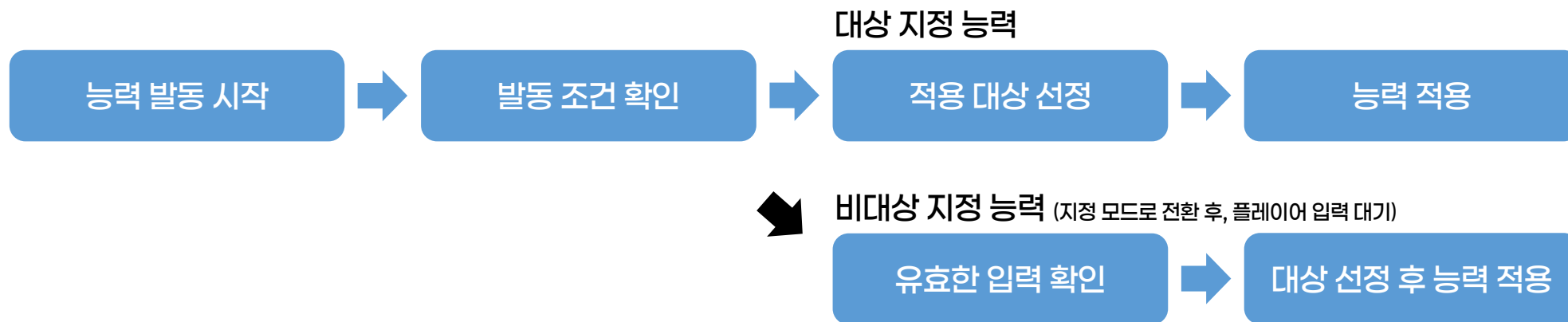
지정한 Slot에 2 피해를 주는 효과



핵심 로직 구현 및 리팩토링 (Ability & Resolve Queue)

대상 지정 능력 vs 비대상 지정 능력

- 대상 지정 능력: 능력 발동과 실제 적용 과정을 분리
- 비대상 지정 능력: 능력 발동과 적용을 모두 한 번에 처리



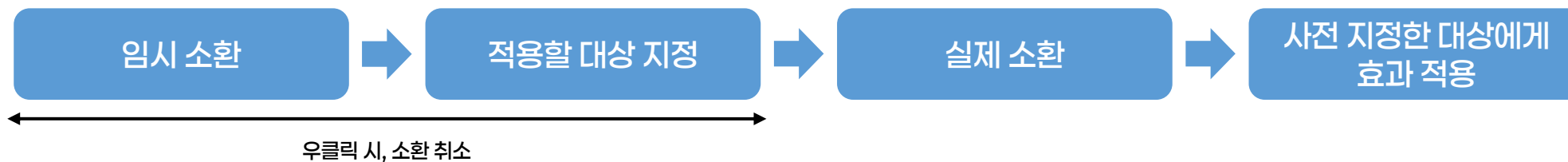
핵심 로직 구현 및 리팩토링 (Ability & Resolve Queue)

Repeat Condition

- 능력 발동 시, 능력의 최대 발동 횟수, 및 현재 반복 횟수를 매개변수로 전달하여 능력을 반복하여 발동할 수 있음
- bool 값으로 반환하여, 단순히 횟수 반복이 아닌 다양한 반복 조건을 설정할 수 있음

대상 지정 효과의 취소

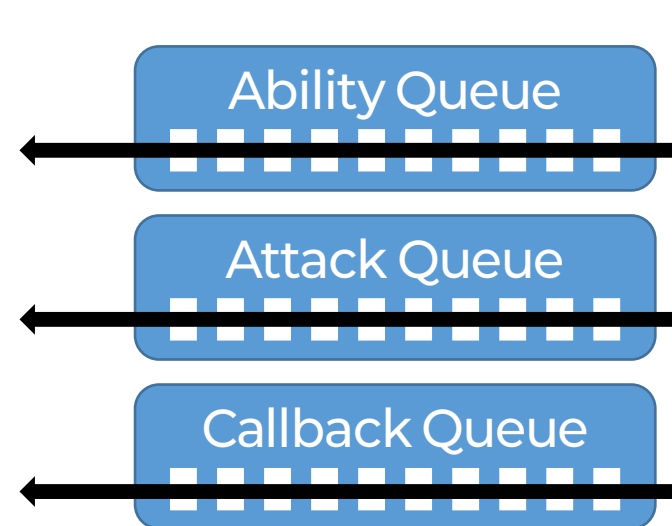
- 능력 발동 후, 대상을 지정하기 때문에 대상 지정 효과의 경우 취소 불가능이 기본 원칙
- '하스스톤'에서는 '전투의 함성' 능력을 취소하여 유닛 소환을 취소할 수 있음
- 소환 후, 능력 발동 구조를 변경하여, 우클릭 시 효과를 취소할 수 있는 기능을 추가



핵심 로직 구현 및 리팩토링 (Ability & Resolve Queue)

Resolve Queue

- 능력의 순차적 발동을 위해 구현된 시스템
- 목적에 따라 3개의 큐로 구분하여 순차적 진행
 - Ability Queue : 능력의 발동을 저장하는 큐
 - Attack Queue : 공격 순서 보장 및 진행하는 큐
 - CallbackQueue : 페이즈 전환과 큰 전환을 담당하는 큐
- 큐의 순서로 1차 순서 보장 및 각 큐를 통해 2차 순서 보장



ResolveAll()

능력이 발동한다 → 효과를 바로 적용하는 것이 아닌, Resolve Queue에 **적재**하는 것 (Enqueue)

적재된 능력은 ResolveAll() 함수를 통해 적용할 수 있음

단일 효과 여러 개를 적재한 후, ResolveAll()을 통해 한 번에 Dequeue 하여 광역기를 적용할 수 있음

핵심 로직 구현 및 리팩토링 (Attack Phase)

개요

플레이어가 차례 종료 버튼을 누르면, AttackPhase로 이동

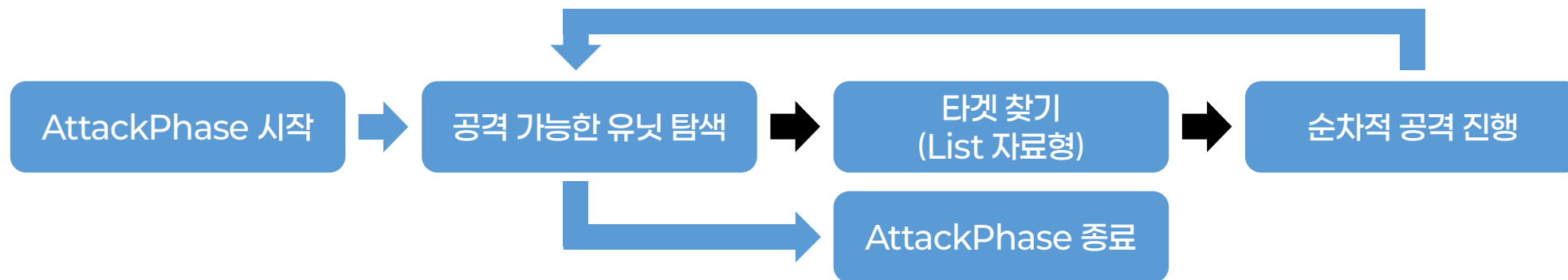
필드의 모든 유닛이 각자의 무기 타입에 맞게 공격을 진행한 후, 상대방 플레이어에게 차례를 넘김

주안점

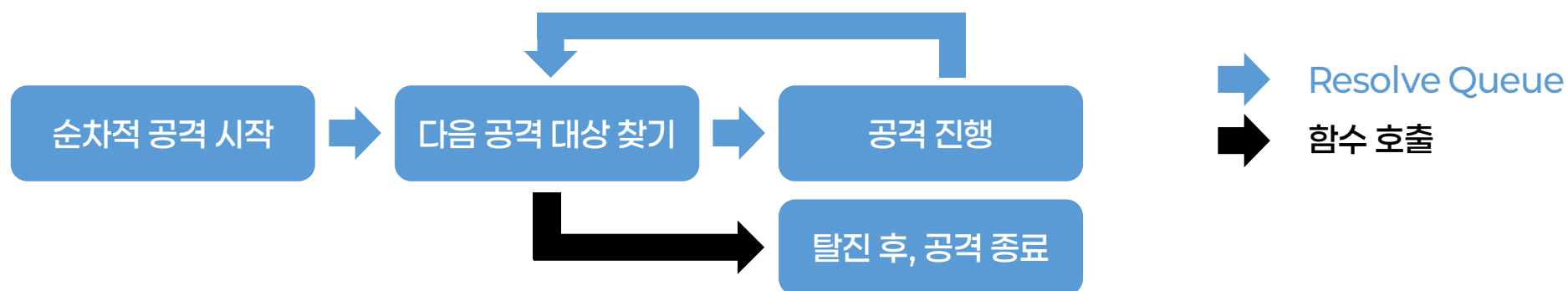
- 공격 시 발생하는 효과를 순차적으로 누락 없이 처리
- 무기 타입 별로 서로 다른 알고리즘으로 공격하도록 유연한 설계 (ex. 사거리 내 무작위, 사거리 내 체력이 가장 낮은 적)

핵심 로직 구현 및 리팩토링 (Attack Phase)

Attack Phase 진행 순서



순차적 공격 진행 순서



핵심 로직 구현 및 리팩토링 (Attack Phase)

공격 진행 순서



Resolve Queue vs 함수 호출

- Resolve Queue : 진행 과정 중 효과가 발생하거나, 하나의 단계가 끝나고 다음 단계로 넘어갈 경우
- 함수 호출 : 효과가 동반되지 않거나, 단계 진행 중 가독성 및 유지보수를 위해 분리한 함수를 실행하는 경우

핵심 로직 구현 및 리팩토링 (Attack Phase)

무기 타입

- 부모 클래스로 Weapon Data 추상 클래스 사용
- 각 자식 클래스를 Scriptable Object로 에셋화 가능하도록 설정 뒤, 카드 데이터에 할당
- 타겟 탐색 및 공격 알고리즘을 자식 클래스에서 구현하여, 타입 별로 다른 알고리즘으로 동작하도록 설계

설계 기대 효과

- 무기 타입 별로 사거리나 UI 색상, 공격 애니메이션 설정이 용이함
- 무기 타입에 대해 분기를 나누지 않아도, 자동으로 각 타입에 맞는 알고리즘 실행
- 새로운 무기 타입 추가 시, 기존 코드를 수정할 필요가 없음

Weapon Data
+ id
+ weapon_tpye
+ range
+ weapon_color
+ SearchTarget(GameLogic, Card): List<Card>
+ AttackTarget(GameLogic, Card, List<Card>)
+ AttackTarget(GameLogic, Card, Player)

WeaponData 추상 클래스

UI / UX 개편

개요

카드 게임의 다양한 UI / UX 및 연출 작업

주안점

- 모듈화를 통해, 특정 클래스에 종속되지 않고, 독립적으로 동작할 수 있도록 설계

UI / UX 개편

Slot Indicator 클래스

플레이어의 특정 행동에 따른, 적절한 정보 표시 (ex. 호버링 시, 사거리 표시)

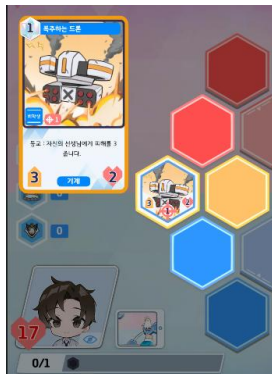
모드를 활용한 '전략 디자인 패턴' 사용

추상 클래스를 통해, 자식 클래스로 모드를 구현

Manager에서 상황에 맞게 모드를 바꾸면, 해당 모드에 맞는 UX가 동작



Before



After

```
private BSlotIndicatorType SetCurrentType(Game game_data, BSlotIndicatorType current_type)
{
    if (GameUI.IsUIOpened())
        return new BSlotIndicatorTypeNone();

    HandCard hcard = HandCard.GetDrag();

    if (game_data.selector == SelectorType.SelectTarget)
        return new BSlotIndicatorTypeSelector();

    if (hcard != null)
        return new BSlotIndicatorTypeDragCard();

    if (current_slot != null)
    {
        Card current_hovering_unit = game_data.GetSlotCard(current_slot);

        if (current_hovering_unit != null && current_hovering_unit.IsHovering())
            return new BSlotIndicatorTypeHoverUnit();
    }

    return new BSlotIndicatorTypeNone();
}
```

상황에 맞게 Mode 설정

UI / UX 개편

마나 커브 및 덱 인포 UI

덱 에디터 환경에서 설정 버튼 클릭 시, 덱 인포 UI로 이동

UI 클래스에 덱 정보를 같이 전달하여, 독립적으로 동작할 수 있도록 설계

아로나 능력 또한, 닫기 버튼을 누를 때, 리턴하여 외부에서 변경된 값을 반영



인게임 마나커브 이미지

```
public void Refresh(List<UserCardData> deck_info)
{
    if (deck_info == null)
        return;

    mana_distribution.Clear();

    foreach (UserCardData utid in deck_info)
    {
        CardData ucard = CardData.Get(utid.tid);
        int mana = ucard.mana < 10 ? ucard.mana : 10;
        if (mana_distribution.ContainsKey(mana))
            mana_distribution[mana] += utid.quantity;
        else
            mana_distribution[mana] = utid.quantity;
    }

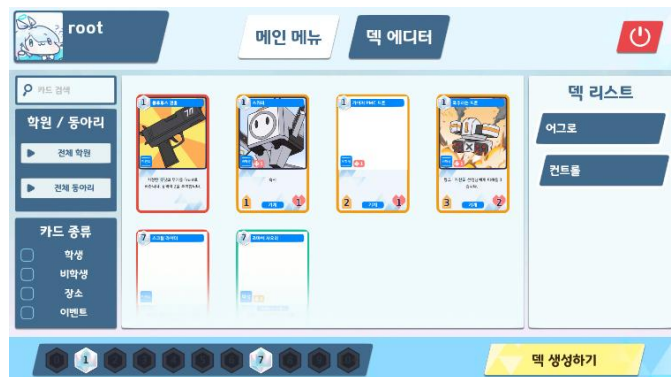
    DrawManaCurve();
}
```

매개변수로 덱 정보를 같이 받아 독립적으로 동작하는 마나커브

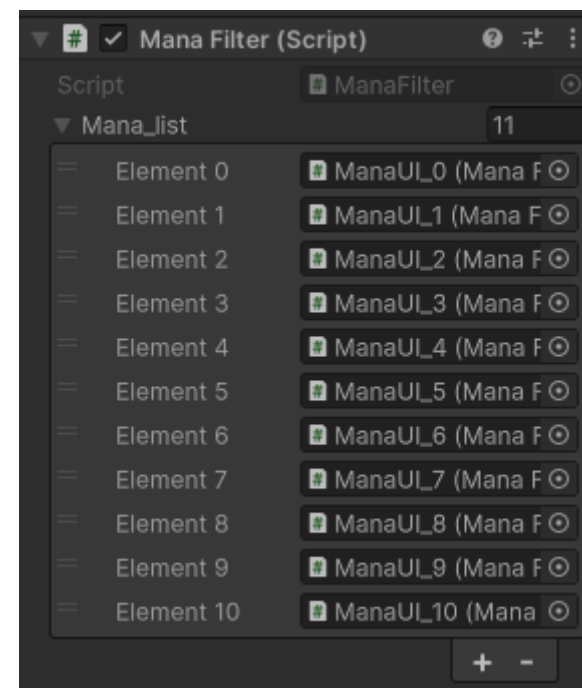
UI / UX 개편

마나 필터링 시스템

옵저버 디자인 패턴을 사용하여, 관리 Manager와 하위 버튼들로 구성
버튼이 클릭되면, 옵저버 클래스에 전달
Manager는 현재 상태와 눌린 버튼의 인덱스를 통해 마나 필터링 진행



인게임 마나 필터링

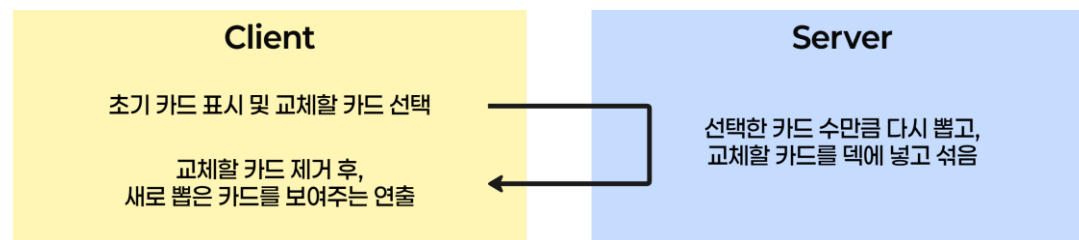


Manager가 관리하는 마나들

기타

멀리건 시스템

1. 게임 시작 시 Phase를 추가하여 멀리건 진행
2. 각 클라이언트가 교체할 카드를 고르면, 해당 카드 리스트가 서버로 전송
3. 서버에서는 새로 뽑은 카드를 클라이언트에 전달
4. 클라이언트는 애니메이션을 통해 교체 연출
5. 서버는 사전에 세팅한 시간이 흐르면 게임 시작



멀리건 시스템 흐름도



멀리건 페이지

```

API authentication: True (1)
Client Connected: 1
Client Connected: 2
Client Disconnected: 2
Client Disconnected: 1
Client Connected: 3
Client 3 connecting to game: gg8jWWtQ8Cwv
Client Connected: 4
Client 4 connecting to game: gg8jWWtQ8Cwv
Match Started: false
[Mulligan] Player 1 replaced [Kinugawa_Kasumi] with [Shimokura_Megu]
Match Completed: false
Client Disconnected: 4
Client Disconnected: 3
  
```

멀리건 데이터 로그

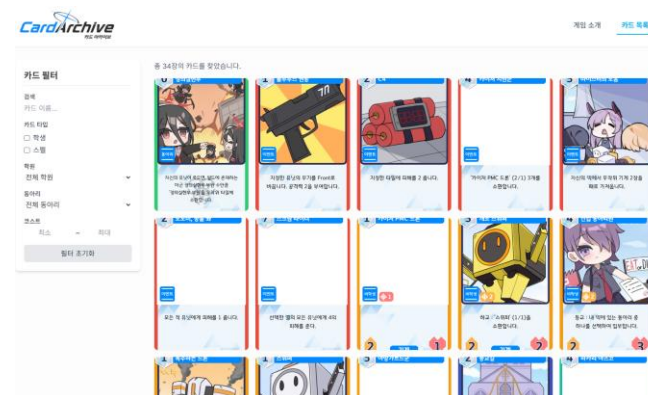
기타

홈페이지 구현 (클릭 시 이동)

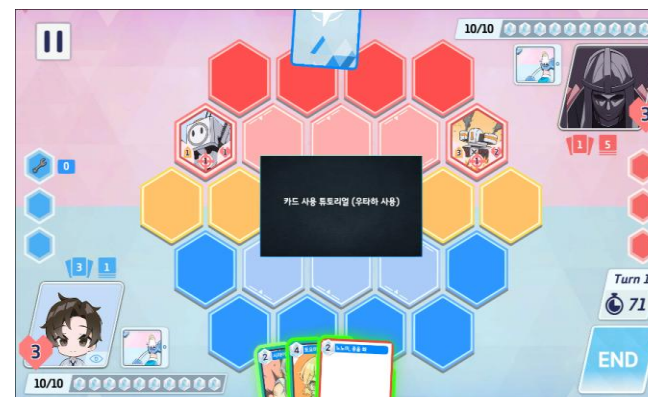
Node js의 express를 활용하여 웹 페이지 서비스
유니티에서 **WebResponse**를 통해 카드 정보와 이미지를 업로드
데이터베이스에 저장된 정보와 이미지를 통해 카드 조회 시스템 구축

튜토리얼 구현

게임 시작 시, 튜토리얼 여부 확인 후 활성화
게임 로직을 건드리는 것이 아닌 플레이어 입력에 관여하여 캔슬시킴
싱글톤으로 구현된 단일 튜토리얼 매니저에 여러 클라이언트 클래스 접근
플레이어의 입력이 현재 단계에서 요구하는 입력과 일치하는지 확인
일치하면 입력 수행 후, 튜토리얼 다음 단계로 진행



게임 배포와 카드 정보를 공개하기 위한 홈페이지



튜토리얼 플레이

성과 및 향후 계획

일러스타 페스 출품

10월 3~4일에 열린 일러스타 페스 인디게임 플레이존 출품

양일 총 124명이 시연

설문조사 후 받은 피드백을 통해 지속적으로 개선

향후 계획

- 2025년 4분기에 배포 후 서비스 진행
- AI 개선 후, 싱글 콘텐츠 추가



일러스타 페스 부스 사진

14:01

100

Thank ***You***
감사합니다